

Kalakriti

Craft, and the market it could never reach.

What is built, what genuinely works, what does not work and why, and what each of the six of us picks up on Monday.

We have a real Android app. It is installed, it opens, and it does the hard part - it makes a listing with no network at all. That is further than most teams get.

We also cannot log in. Not one of us, on any machine, because no SMS provider is configured and the development bypass is switched off. And nothing we have built is reachable from outside one Wi-Fi network, because every listing URL points at a private address.

Both of those are true at the same time, and neither is a reason to panic or to start again. This document separates the two so we stop guessing which is which.

Three things this document settles

1. Exactly what is working, with the command that proves each line - so nobody rebuilds something that already works.
2. That we are building a **mobile app**, not a website that happens to open on a phone. What that rules out, and how we know when a feature is actually done.
3. Why we did not use Flutter or Kotlin, whether that was the right call, and the honest answer to whether we should switch now. Short version: we are not switching, and section 5 shows the working.

START HERE

The sixty-second version

9,431 lines of TypeScript in the app	7,120 lines of Python in the backend	512 x 9 interface strings x languages	1 of 5 sales channels able to publish
--	--	---	---

Read these six lines and you have the whole picture. The rest of the document is the evidence and the plan.

- **The app is real and it runs on a phone.** There is a release APK on disk. Photograph a product with no signal and a draft exists on the device the instant the shutter is pressed. Nine languages render with the aeroplane mode on.
- **The backend is real too.** Seven thousand lines, thirty-four endpoints, a relational schema, real OTP authentication, a public product page with a Buy button that writes a genuine order row. It is not a mock. It runs on one laptop.
- **Nobody can log in right now.** There is no SMS provider configured and the local bypass is off, so the code is generated correctly and then delivered nowhere. This is a five-minute fix locally and a several-day fix properly.
- **Nothing is public.** Every listing URL, image path and webhook target points at 10.154.140.9, a private address. A judge scanning our QR code on their own phone would get nothing at all.
- **Four of our five sales channels cannot publish.** ONDC, GeM, Amazon and Shopify are written and waiting for credentials that need accounts somebody has to open. Our own storefront works today.
- **So the work left is mostly not code.** It is one SMS account, three hosting signups, and paperwork that takes weeks and therefore has to start this week. That is the single most important sentence in this document.

What this means for how we spend the week

The instinct when a project feels incomplete is to write more features. That instinct is wrong here. We are not short of features. We are short of an SMS account, a hosted database, a public URL and four seller registrations.

If every one of us wrote code all week and nobody opened those accounts, we would end the week with a better app that still cannot be logged into or reached. Section 6 puts the accounts first on purpose.

SECTION 1

What is built

Two pieces and one artefact. It helps to be precise about which is which, because "the app" gets used for all three in conversation.

Piece	What it is	State
app/	Expo / React Native, TypeScript. The Android app, plus a browser build used only for iteration.	Typechecks clean. Release APK built.
backend/	Python and FastAPI. Vision, OCR, pricing, copywriting, auth, orders, the public storefront, marketplace adapters.	Imports clean. Runs on one laptop.
The APK	app/android/app/build/outputs/apk/release/app-release.apk	Exists, installs, runs. 113 MB - see section 3.

Both health checks pass today, and they are the two commands to run before every commit:

```
cd app && ./node_modules/.bin/tsc --noEmit # silent = good
cd backend && python -c "import main" # imports cleanly
cd backend && python preflight.py # the real status report
```

That third command is the one that matters. **preflight.py** does not trust documentation - it connects to the database, calls the language model, checks every credential and tells you which variable is missing. Section 3 of this document is its output, written out in sentences. When this PDF and preflight disagree, preflight is right and this PDF is stale.

What the app does, end to end

The artisan journey is built and connected. Bottom tab bar: Home, My Products, Orders, Profile, with the create flow pushed over the top of it.

Step	What happens	Needs network?
Onboarding	Choose one of nine languages. Persisted on the device and later synced to the profile.	No
Capture	In-app viewfinder with live framing, exposure, blur, tilt and clutter guidance, plus a per-craft shot plan.	No
Draft	The photograph is copied into the app's own storage and a draft exists immediately. It appears in My Products.	No
Describe and price	Auto mode runs a server job on the whole listing; manual mode puts an AI button on individual fields with a Keep mine option.	Yes
Publish	Channel list, seller readiness check per marketplace, then publish.	Yes
After the sale	A public product page, real order rows, bulk and custom order enquiries, per-channel metrics.	Yes

SECTION 2

What works today

Do not rebuild anything on this list. Each line was verified by running something, not by reading a file. If you think one of these is broken, say so in the group with the steps - it means either a regression or a wrong line here, and both matter.

Capability	State	How we know
Listing creation with no signal	WORKS	Press the shutter in aeroplane mode: a draft is written to the device, shows in My Products, opens where it was left, and uploads oldest-first when a backend answers. This was the last commit and it is the most important thing the app does.
Guided camera	WORKS	Live framing, exposure and glare, blur by Laplacian variance, tilt by gradient orientation, clutter ratio, subject box from edge-energy profiles. Classical computer vision in TypeScript on a 160x120 buffer, so it needs no model download, no Play Services and no network.
Nine languages, offline	WORKS	512 interface strings, English and Hindi authored by hand, seven more generated and committed as JSON. Fallback is chosen language, then English, then the key - never Hindi, so a gap looks like a gap rather than a broken switch.

Capability	State	How we know
Persistence across a kill	WORKS	Session token in the Android Keystore through SecureStore; language, cache and outbox in AsyncStorage. First render is gated on storage being read back, so the app never flashes the language picker at somebody who chose Gujarati weeks ago.
Offline edits, no lost work	WORKS	Edits apply locally at once and queue. Repeated edits to one row collapse into one operation. On replay the server resolves per field from an append-only event log: when two people changed different fields, both changes survive.
OTP authentication	WORKS	Random code, salted SHA-256, expiring, rate-limited, server-side sessions that logout genuinely revokes. Verified end to end against a real number: wrong code 400, correct code signs in, reusing the code 400, logout kills the session.
The public storefront	WORKS	//{id} is a real server-rendered product page with a working Buy button that creates a genuine order row. It is the one channel that needs nobody's approval, and it is fully real. It is also only reachable on our Wi-Fi - section 3.
Honest metrics	WORKS	Storefront views counted from real requests, viewer reduced to a salted hash bucketed hourly. Every metric a platform does not expose returns available:false with the platform's own reason, and the app renders a dash you can tap. Never a zero.
Two listing modes	WORKS	Auto mode queues a server-side job over the whole listing; manual mode gives each field its own AI button. Both were exercised. Note the caveat in section 4: they were verified in a browser, not on a phone.
The language model	WORKS	preflight accepts the NVIDIA key and completed a round trip to Nemotron. It took 14 seconds for one token, which is a latency question nobody has measured on the real create-flow path yet.

The part of this list that is genuinely hard to copy

Any team can put a form in front of a camera. Three things here are not easy and they are what we should be showing: **a listing that completes with no signal, camera guidance that works with the aeroplane mode on, and metrics that admit what they do not know.**

That last one sounds like modesty and is actually the strongest thing we have. Every other cataloguing demo shows a confident number. Ours says "Amazon does not expose this without Brand Registry" and lets the artisan tap to find out why. A zero says nobody looked; the truth is nobody counted, and those are different claims.

SECTION 3

What does not work, and exactly why

This is the section to read twice. Every line comes from running **backend/preflight.py** or from inspecting the build, not from memory. The run produced two passes, six warnings, one failure and one skip.

The three that stop a demo

What	State	Why, and what it costs us
Login, for everybody	BLOCKER	No SMS provider is configured and OTP_DEV_ECHO is off, so sms.py returns not_configured and the code is delivered nowhere. The code itself is correct and still has to match - nothing is weakened - but no login can complete in any environment, including on our own laptops . Local fix today: set OTP_DEV_ECHO=true. Real fix: an MSG91 account with a DLT-approved template, which takes days to clear.
Anything being public	BLOCKER	PUBLIC_BASE_URL is http://10.154.140.9:8000, a private address. Listing URLs, image paths and webhook targets are all built from it, so only a device on that one Wi-Fi can open them. Correct while building against a laptop. Useless the moment we hand a link or a QR code to anyone.
The database is one file on one laptop	BLOCKER	DATABASE_URL is sqlite:///./kalakriti.db, currently 8 MB. On a laptop that is a real relational database and it is fine. Deployed to a container it is silent data loss: the filesystem is replaced on each deploy, the tables are recreated empty, no error is raised, and the first person to notice is an artisan who can no longer log in. The code already reads Postgres - only the variable is unset.

The ones that will embarrass us

What	State	Why, and what it costs us
The APK is 113 MB	BROKEN	The release APK on disk is 113 MB, built with all four CPU architectures and no minification. Our README claims 32 MB and says the shrink flags are already set in android/gradle.properties - they are not there . Our user is on an entry-level handset with metered data. Of everything on this page, this is the defect that most contradicts the entire argument of the product.
Product images on local disk	BROKEN	No object storage is configured, so images are written under MEDIA_DIR on the container filesystem. Slightly worse than the database problem, because a marketplace will already have published the image URL - so after the next deploy the listing stays live with a dead photograph.
Everyone silently logged out on deploy	BROKEN	JWT_SECRET is unset, so auth.py generates a key and writes it to jwt_secret.txt. That file goes with the container filesystem, so every session token signed with the old key stops verifying and everybody is logged out with no error anywhere.
Four of five channels	NOT BUILT	ONDC, GeM, Amazon and Shopify all report not_configured and name the exact variables they want. The integration code is written and real - signed Beckn payloads, SP-API with LWA token exchange, Admin GraphQL. What is missing is accounts and approvals, which is paperwork, not engineering.
Odia and Telugu	PARTIAL	Of 512 strings, 92 in Odia and 52 in Telugu are still identical to English. The language switch works perfectly; the translation behind it is incomplete. An Odia speaker sees roughly one screen in five in English.

What	State	Why, and what it costs us
Couriers	NOT BUILT	No courier is configured. Orders are still created and tracked in the database. What is absent is pickup booking and a tracking number, which the seller would have to arrange herself.

The quieter ones, so nobody is surprised

What	State	The situation
iOS	NOT BUILT	There is no app/ios directory and no iOS build has ever been made. Android only. This is a reasonable decision for our user and our timeline - it is only a problem if somebody assumes otherwise in a demo.
Push notifications	NOT BUILT	expo-notifications is not a dependency. When a server job finishes a listing, the app shows a card the next time it is opened and a count on the Home tab. That is real and it works - it is just not a notification, and we should not call it one.
The README	BROKEN	It points at app/src/lib/ai.ts, which no longer exists, and describes a mock AI layer that the real backend replaced. It says 330 strings where there are now 512, and 32 MB where the build produces 113 MB. Anybody onboarding from it will be confused, and a judge reading it will find claims the repository contradicts.
Model latency	HEADS UP	A one-token round trip to Nemotron took 14 seconds. That number on its own is not alarming, but nobody has measured what the create flow actually feels like on a phone on mobile data. Somebody should, before the demo, not during it.
VIEW_SALT	HEADS UP	Unset, so it is derived from JWT_SECRET. View rows are still hashed rather than storing an address, so nothing is leaking. The cost is that rotating JWT_SECRET also resets every unique-view count to zero.

How to read the difference between these two sections

Nothing in section 3 is a fake feature or a lie in a slide. There is no mock data pretending to be real. Every item is one of three things: a variable that is not set, an account nobody has opened, or a build flag missing from one file.

That is a genuinely good position to be in eight days out. It is also only good if we act on it this week, because the account approvals are the part we cannot compress.

SECTION 4

This is an app, not a website in a phone-shaped window

We have a browser build and it has earned its place - it is much faster to iterate in. But it is not the deliverable, and it must never be the thing we demo or the thing we call done.

This is not pedantry. The entire argument of this product is that it works for somebody on an entry-level Android phone with bad light and unreliable data. Every single claim in that sentence is untestable in a browser on a laptop.

What only exists on a real device

- **The camera.** The in-app viewfinder, the guidance overlay and the permission flow. GuidedCamera.native.tsx and GuidedCamera.web.tsx are literally different files, and only one of them ships. Testing one tells you nothing about the other.

- **The keystore.** On a phone the session token sits in the Android Keystore. On web, SecureStore has no implementation and falls back to localStorage. A browser tab has no keystore, so the security property we designed for does not exist there.
- **Offline behaviour.** Aeroplane mode, the outbox draining when the app is brought back to the foreground, a draft surviving the process being killed. A browser tab does not get killed the way Android kills an app.
- **The physical realities.** The hardware back button, 56dp touch targets, one-handed reach, whether the palette is legible in direct sunlight, cold-start time on a cheap phone, and what the guidance loop does to the battery.
- **Install size.** 113 MB is invisible in a browser and decisive on a metered connection.

Our own commit history says the quiet part out loud

One of our commits is titled "*Verify the two listing modes in a browser*". That was an honest message and the work was real. It is also exactly the gap this section is about: a browser could not tell us whether either mode is usable on a phone, and we still do not know.

So the two listing modes are in section 2 as working, with that caveat attached. They are not yet done under the definition below.

The definition of done, from now on

A feature is done when it works from the installed APK, on a real mid-range Android phone, with the laptop closed - and with no data at all if it claims to work offline.

Not done: it works in Expo Go on the same Wi-Fi. Not done: it works in Chrome. Not done: it works with adb reverse tunnelling a laptop address back over a USB cable.

Three rules that follow from it

1. **Never bake a laptop address into a build anybody else will hold.** We already had this bug: 127.0.0.1 was compiled into an APK and appeared to work only because a USB cable was tunnelling it back to the laptop. On a phone, that address is the phone.
2. **If a feature needs a browser affordance, it is not designed yet.** A URL bar, a tab, a desktop file-picker dialog, a hover state, localStorage as the real store. An artisan has none of these and should never be asked to type a URL - we removed a "Server address" field from Profile for exactly this reason.
3. **Every demo runs from a sideloaded APK, on a phone, and it gets sideloaded a day early.** An emulator hides low-end performance, which is the one thing we are claiming to have solved. Everybody on the team should install the current APK and use it for ten minutes before writing another line of code.

SECTION 5

Why not Flutter, and why not Kotlin

This is a fair question and it deserves the real answer rather than a defence. Flutter and Kotlin are the default answers for Android app development, and we used neither. Here is what that bought, what it genuinely cost, and then whether to change course - which is a separate question from whether it was right.

What React Native and Expo bought us

Each of these is load-bearing for this project specifically, not a general preference.

- **One language across the whole team.** The backend is Python and the app is TypeScript. With Kotlin or Dart, everybody carries a third language while also learning a new build system, in a week we do not have.
- **An APK without a local Android toolchain.** EAS builds on Expo's servers and hands back a download link. Getting JDK 17 and a matching Android SDK working on six different machines is a day of the schedule, and it is a day spent on nothing a judge can see.
- **The native modules we needed were already wired.** Camera, secure store, SQLite, speech, localization, haptics, file system, image manipulation. Each of those is a native permission-and-lifecycle problem we did not have to solve.
- **A browser surface for iteration.** Worth real hours during a build week, with the discipline from section 4 attached to it.
- **iOS stays possible** from the same codebase, even though we have not built it and should not this week.

What it cost us - one thing, and it is real

expo-camera exposes no frame-processor callback. This was verified against the installed typings: the only per-frame hook available is barcode scanning. So the guidance loop cannot simply read frames. It samples the preview, downscales it with a native module, and then decodes the JPEG in pure JavaScript on a 160x120 buffer.

That is why the estimators are classical computer vision rather than a model, and why the loop runs at a handful of frames a second instead of thirty. In Kotlin the same loop is CameraX plus ML Kit: real object detection and tracking at roughly 30fps, no JPEG round trip, no JavaScript decode. That is a genuinely better camera experience and we should say so plainly rather than pretend otherwise.

The three-way comparison, against what this project actually needs

	React Native + Expo (<i>chosen</i>)	Kotlin (<i>native</i>)	Flutter
Language for this team	TypeScript, already known	Kotlin, new to all of us	Dart, new to all of us
APK with no local SDK setup	Yes, cloud builds	No	No
Real-time frame analysis	Weakest. No frame API; needs VisionCamera and a dev build	Best. CameraX + ML Kit, ~30fps	Middle. Frames available, ML via platform channels
On-device ML (ML Kit, MediaPipe)	Native module or community wrapper	First-class	Plugin or platform channel
Nine-language Indic interface	Working today, 512 strings	Would be rebuilt	Would be rebuilt
Offline queue and per-field conflict resolution	Written and working	Would be rebuilt	Would be rebuilt
Browser surface for iteration	Free, same code	None	Yes, but heavy

	React Native + Expo (<i>chosen</i>)	Kotlin (native)	Flutter
iOS later	Same codebase	A second codebase	Same codebase

So was it a good idea? And should we rotate?

I want to answer this with a test rather than an opinion, because "which framework is better" has no answer and "which framework gets us to a working demo" does.

The test: list the work that is actually left, then count how much of it a rewrite would fix.

Work still left	Would Kotlin or Flutter fix it?
OTP delivery, so anyone can log in	No. An SMS account and a DLT template.
A hosted database instead of a laptop file	No. Backend configuration.
Object storage for images	No. Backend configuration.
A public HTTPS host	No. Hosting.
Secrets that survive a deploy	No. Two environment variables.
ONDC, GeM, Amazon and Razorpay credentials	No. Registrations and approvals that take weeks.
92 Odia and 52 Telugu strings	No. A translation run.
Bringing the APK from 113 MB down	No. Two Gradle flags. Both platforms have the same problem to solve.
Multi-photo capture completing the shot plan	No. Product work in our own code.
Progressive delivery on a bad connection	No. Product work in our own code.
Faster, smarter camera guidance	Yes - and this is the only one. See the escape hatch below.

The verdict: it was the right call, and we are not rotating

Ten of the eleven things standing between us and a working demo are not framework problems. They are backend configuration, hosting, paperwork and product work. A rewrite fixes none of them.

What a rewrite would do is spend the entire remaining schedule reproducing what already works - the nine languages, the offline drafts, the outbox with per-field conflict resolution, the storefront - and arrive at exactly the same list of blockers, minus a working APK. We would be strictly worse off and later.

The framework is not what is standing between us and a demo. An SMS account and a hosting signup are.

The one place native is the right answer

If the camera guidance turns out to be too slow or too vague on a real phone, the fix is still not a rewrite. It is **react-native-vision-camera** with a frame processor running on a worklet thread, which wraps the same CameraX a Kotlin app would use, plus ML Kit through a native module. Our own docs/ON-DEVICE.md already names this as the upgrade path and explains why it was deferred.

That change is bounded inside the capture module's files. It keeps the languages, the offline queue, the storefront and the APK pipeline. It is the only place in this project where paying for native is worth it.

A decision rule, so we stop re-litigating this every few days

If somebody says...	The answer is
"The camera guidance is slow or imprecise"	A VisionCamera frame processor, inside the existing app. Capture module owns it.
"The app is slow to open, or too big"	Build configuration: architectures, minification, shrinking. Not a framework.
"Expo has no module for the thing we need"	Write one native module. Still inside the existing app.
"We should have used Flutter or Kotlin"	Name a blocker from section 3 that switching would fix. If there is one, we talk. There currently is not.

SECTION 6

What each of us does next

Module numbers are the same as in [Kalakriti_Team_Modules.pdf](#). The accounts come first because approvals are the only thing on this page that we cannot make go faster by working harder.

Everyone, today, fifteen minutes

1. Set `OTP_DEV_ECHO=true` in `backend/.env`, so you can log in at all. Nobody can currently sign in on any machine, and half the app is behind a login.
2. Run `python preflight.py` and read the whole output. It is the source of truth for section 3, not this document.
3. Install the current APK on a real Android phone and use it for ten minutes. Every one of us, before writing another line. Section 4 is why.
4. Run the two health checks before every commit. Both are clean today and keeping them clean is much cheaper than fixing them later.

Then, by module

Owner	Do this first	Done when
M3 Identity	Open the MSG91 account and submit the DLT template today . Approval takes days and it blocks every login in the product. Until it lands, everyone uses dev echo locally.	A code arrives on a phone that is not yours, with <code>OTP_DEV_ECHO</code> switched off.
M5 Platform	Hosting, in this order: a Postgres database, object storage for images, then the backend on a public HTTPS host. Then set <code>PUBLIC_BASE_URL</code> , <code>JWT_SECRET</code> and <code>VIEW_SALT</code> , and rebuild the APK against the https address.	<code>preflight.py --production</code> exits zero, and a listing opens on a phone using mobile data with the laptop shut.
M5 Platform	Fix the APK size: restrict to arm64-v8a and add the minify and shrink-resources flags to <code>android/gradle.properties</code> , which currently do not contain them.	The release APK is comfortably under 40 MB and still installs and runs on a real phone.
M5 Platform	Refill the 92 missing Odia strings and 52 Telugu ones, then run the script check to catch mixed-script output.	No language has more than a handful of strings identical to the English source.
M4 Commerce	A free Shopify development store issues an Admin API token in minutes, with no onboarding queue. It is the one external marketplace we can genuinely have live this week.	A product created in our app is live on a real external store, and an order placed there appears in Orders.

Owner	Do this first	Done when
M1 Capture	Multi-photo capture that completes the shot plan and gets every photograph onto the listing, plus a quality gate that refuses a shot too blurred or too dark and says which.	A pot photographed front, top and base shows three images on the product page, and a deliberately blurred shot is rejected with a reason.
M2 Listing studio	Progressive delivery on a bad connection: fields first, then a thumbnail, then the full image, each only when the connection allows.	On a throttled connection the listing is editable before any image has arrived.
Whoever has an hour	Start the paperwork that cannot be compressed: ONDC network-participant registration (weeks), GeM seller account (days to weeks), Amazon Professional account and LWA app (days), Razorpay KYC (days).	Each one is submitted and waiting. None of this is code, and all of it blocks a channel.
Anyone, half an hour	Correct the README. It names a file that no longer exists, describes a mock layer we replaced, and claims an APK size the build does not produce.	Nothing in the README is contradicted by the repository. A judge will read it.

SECTION 7

The two rules that keep this honest

Everything else in this document will be out of date within a week. These two will not.

1. No module invents a number

If a platform does not expose a metric, return `{value: null, available: false, why: "..."}` and let the app render a dash with the reason behind it. Never a zero, and never an estimate dressed as a measurement.

A zero says nobody looked. The truth is that nobody counted, and those are different claims. An artisan deciding whether a channel is worth her time deserves to know which one she is being told.

We already removed a feature for breaking this rule: a screen that asked a language model for "three trends with a realistic rupee band" and showed them with a confidence percentage. A model guessing what handloom sells for is not market data. This is the thing judges will probe hardest, and it is what separates us from every other cataloguing demo.

2. Nothing is working until it works from the APK with the laptop shut

A browser tab is not a phone. Expo Go on our Wi-Fi is not a phone. An emulator is not a phone, and it hides exactly the low-end performance we are claiming to have solved.

Section 4 has the full definition. The short version: install it, unplug it, walk away from the desk, and then decide whether it works.

Where we actually stand

We are further along than the blocker list makes it feel. There is a real app on a real phone doing the genuinely hard thing - building a complete listing with no network - in nine languages, with a backend behind it that does not fake its data.

What is missing is almost entirely on the other side of a signup form. One SMS account, three hosting services, four seller registrations and two Gradle flags. The parts that take weeks are the parts we start today.

So the ask is small and specific: **open your account before you open your editor**. And if this document and preflight.py ever disagree, preflight is right and this document is stale.